Documentação técnica da API

Projeto do Hackathon | TradeHub | CRUD com JSON Server

Documentação da entidade Produto, adaptada à implementação de cadastro, consulta, edição e exclusão de anúncios com persistência no arquivo db.json.

1 Identificação

Informação	Preenchimento
Nome do projeto	TradeHub
Turma e equipe	Turma não informada / TradeHub
Repositório	https://github.com/Hackathon-Trade-Hub/Trade-hub
Entidade documentada	Produto - coleção produtos
Integrante responsável	Pedro Schetz
Versão e data	1.0 / 11/11/2026

2 Finalidade da entidade

Representar os produtos anunciados para venda, troca ou ambas. O cadastro grava um registro no db.json; a edição atualiza o registro existente; a exclusão remove o registro. A listagem na interface é atualizada após a confirmação da API.

3 Campos e regras

Campo	Tipo	Obrigatório	Descrição ou regra
id	Texto na API	Automático	Gerado pelo JSON Server no cadastro. A aplicação converte IDs numéricos para número.
titulo	Texto	Sim	Título do anúncio; formulário obrigatório com remoção de espaços nas extremidades.
preco	Texto	Sim	Valor maior que zero, validado e formatado em reais, como R$ 250,00.
categoria	Texto	Sim	Categoria selecionada entre as opções do formulário.
status	Texto	Sim	Venda, Troca ou Troca/Venda.
descricao	Texto	Sim	Descrição obrigatória do produto.
imagem	Texto	Sim	Imagem existente ou imagem selecionada, reduzida antes do envio.
vendedorId	Texto	No cadastro	ID da conta conectada que publicou o anúncio.
vendedorNome / vendedorFoto	Texto	Da conta	Nome e foto obtidos da conta conectada.

As validações pertencem à aplicação. O JSON Server não aplica automaticamente essas regras nem autentica o vendedor.

Endpoints e exemplos

4 Endpoints da entidade Produto

URL base: http://localhost:3000
Coleção no db.json: produtos

Método	Endpoint	Comportamento
GET	/produtos	carregarProdutos busca a coleção e preenche listaProdutos.
GET	/produtos/:id	Disponível no JSON Server. As telas atuais consultam o produto na lista já carregada.
POST	/produtos	adicionarProduto envia os dados e insere o produto retornado na lista.
PATCH	/produtos/:id	atualizarProduto envia as alterações e substitui o item da lista pela resposta.
DELETE	/produtos/:id	removerProduto exclui na API e depois retira o item da lista.

Atualização escolhida: PATCH. Permite atualizar os campos enviados e preservar os demais, como os dados do vendedor. O formulário utiliza PATCH; não utiliza PUT. Substitua :id pelo identificador real retornado no cadastro.

5 Corpo das requisições

POST /produtos - exemplo fictício. Cabeçalho: Content-Type: application/json. O cadastro não envia um ID fixo.

```
{
  "titulo": "Bicicleta usada",
  "preco": "R$ 250,00",
  "categoria": "Esportes",
  "status": "Venda",
  "descricao": "Bicicleta em bom estado de conservação.",
  "imagem": "/images/bicicleta-exemplo.png",
  "vendedorId": "usuario-exemplo",
  "vendedorNome": "Pessoa Exemplo",
  "vendedorFoto": ""
}
```

A imagem acima é apenas ilustrativa. No formulário, a imagem selecionada é reduzida e enviada como texto em uma URL de dados.

PATCH /produtos/ID_DO_PRODUTO - exemplo de alteração parcial:

```
{
  "preco": "R$ 230,00",
  "descricao": "Preço atualizado. Produto bem conservado."
}
```

GET e DELETE não precisam de corpo JSON neste fluxo.

6 Relacionamentos

vendedorId vincula o produto ao id de uma conta da coleção usuarios. Carrinho e favoritos utilizam produtoId para referenciar produtos. A exclusão atual remove o anúncio, sem limpeza automática desses vínculos. Não foi implementado bloqueio de exclusão por relacionamento.

A aplicação verifica o vendedor antes de editar ou excluir e exige login nas rotas de gerenciamento. Essas verificações ocorrem no cliente; não representam controle de acesso no servidor.

Validações e comportamento

7 Telas e comportamento esperado

Tela ou ação	Implementação
Listagem	src/main.js chama carregarProdutos ao iniciar. A coleção vem de GET /produtos. Um banco vazio permanece vazio, sem recriar anúncios de demonstração.
Cadastro	src/Views/CadProduto.vue valida os campos e chama adicionarProduto. Só redireciona ao perfil após a confirmação da API.
Detalhes	A aplicação localiza o anúncio na lista carregada, comparando o ID da rota com o ID do produto.
Edição	O mesmo formulário preenche os dados existentes e chama atualizarProduto com PATCH. A rota verifica se a conta corresponde ao vendedor.
Exclusão	O primeiro clique solicita confirmação no botão; o segundo chama removerProduto. Sair sem confirmar não envia DELETE.
Durante a requisição	processando bloqueia os botões de salvar e excluir e evita o envio repetido enquanto a operação está em andamento.
Falhas	Cadastro e exclusão exibem a mensagem de erro no formulário. A lista só muda após uma resposta HTTP bem-sucedida. Falhas na carga inicial são registradas no console.

Seleção de imagem: o arquivo precisa ser uma imagem de até 5 MB; a imagem é reduzida antes de ser persistida. O anúncio exige imagem, preço válido e modalidade de negociação.

8 Códigos de resposta

Código	Significado e tratamento
200	Resposta de sucesso com conteúdo, por exemplo em consultas.
201	Recurso criado, normalmente no POST.
204	Sucesso sem conteúdo. O utilitário retorna null e não chama resposta.json().
400	Requisição inválida, se retornada. As validações de negócio não são automáticas no JSON Server.
404	Registro ou rota não encontrado. É tratado como erro HTTP.
500	Falha interna do servidor. É tratada como erro HTTP.

O utilitário requisitar verifica resposta.ok. Respostas fora da faixa de sucesso geram uma mensagem com o código recebido. Falha de conexão orienta iniciar o servidor com npm run api.

Os códigos acima são referências de interpretação. Nos testes já executados foi confirmado o sucesso das operações, mas os códigos numéricos individuais não foram registrados; por isso não são apresentados como códigos observados.

Testes e entrega

9 Registro dos testes

Verificação automatizada executada com um JSON Server local e um arquivo JSON temporário, sem alterar os produtos reais. A compilação da aplicação também foi concluída com sucesso.

Operação	Verificação realizada	Código observado	Resultado
Cadastrar	Enviar produto e conferir o registro gravado no arquivo.	Não registrado	Aprovado
Listar	Recarregar a coleção e conferir o produto persistido.	Não registrado	Aprovado
Consultar ID	GET individual para ID existente e inexistente.	Não executado	Pendente
Atualizar	Alterar título e conferir o arquivo e a lista após nova carga.	Não registrado	Aprovado
Excluir	Excluir e verificar ausência no arquivo e na lista.	Não registrado	Aprovado
Banco vazio	Recarregar após excluir o último item e conferir que continua vazio.	Não registrado	Aprovado
Permissão	Tentar excluir informando outro vendedor; confirmar rejeição no cliente.	Sem requisição	Aprovado
Conexão	Simular falha de rede e conferir rejeição do cadastro.	Sem resposta HTTP	Aprovado

Execução técnica: verificação automatizada durante a implementação.
Data: 11/11/2026.
Atualização escolhida: PATCH.
Conferência do integrante: Pedro Schetz.

10 Organização da entrega

Como iniciar: execute npm run api em um terminal para servir o db.json na porta 3000. Em outro terminal, execute npm run dev para iniciar o site com Vite. A inicialização permanece separada, conforme solicitado.

JSON Server: versão declarada no projeto ^1.0.0-beta.15. O navegador envia requisições à API; é o JSON Server que grava o arquivo db.json.

Arquivo	Responsabilidade
src/data/produtos.js	requisitar, carregarProdutos, adicionarProduto, atualizarProduto e removerProduto; integração com /produtos e atualização da lista reativa.
src/Views/CadProduto.vue	Formulário de cadastro e edição, validações, confirmação da exclusão e bloqueio de cliques repetidos.
src/router/index.js	Rotas de cadastro e edição, exigência de login e verificação do vendedor na edição.
src/main.js	Carregamento inicial dos produtos antes de montar a aplicação.
package.json e db.json	Comandos de inicialização e arquivo que armazena a coleção produtos.

Esta ficha cobre o CRUD de produtos. Para uso no site publicado, a API precisa estar hospedada e acessível; localhost:3000 atende ao desenvolvimento local. Commits e divisão dos demais CRUDs não foram verificados nesta documentação.